

# Architecture Logicielle — Horosphere

## Jalon 4 — Section : Description de l'architecture multi-couches

### 1. Pattern MVC avec Symfony

Horosphere implémente le pattern **Model-View-Controller (MVC)** en séparant clairement les responsabilités entre trois couches logiques.

#### Contrôleurs (Controller)

Les contrôleurs Symfony constituent le point d'entrée de toute requête HTTP. Ils ont un rôle volontairement limité :

1. Recevoir la requête et vérifier les droits d'accès ( `denyAccessUnlessGranted()` ).
2. Extraire et valider les paramètres de la requête.
3. Déléguer la logique métier au service approprié.
4. Retourner une réponse JSON structurée.

```
api/src/Controller/
├─ AuthController.php      - authentification JWT (login, /me)
├─ PointageController.php  - arrivée / départ avec géofencing
├─ DemandeController.php   - soumission et traitement des demandes
├─ AlerteController.php    - consultation et gestion des alertes
├─ UserController.php      - CRUD utilisateurs (RH / Admin)
├─ SiteController.php      - gestion des zones géofencing
└─ DocumentController.php  - export CSV / PDF
```

Un contrôleur ne contient **jamais** de requête SQL directe ni de calcul métier : il orchestre uniquement.

#### Services (logique métier)

Toute la logique applicative est encapsulée dans des classes du dossier `src/Service` . Chaque service est responsable d'un seul domaine fonctionnel (principe SRP — voir section 3) :

| Service           | Responsabilité                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------|
| PointageService   | Enregistrement des arrivées/départs, détection des anomalies horaires                                 |
| GeofencingService | Calcul de distance Haversine, vérification de la présence dans une zone GPS                           |
| AlerteService     | Création d'alertes typées (OUBLI_DEPART, HORS_ZONE, ECART_HORAIRE), tâche planifiée Symfony Scheduler |
| DemandeService    | Workflow de soumission et de validation des demandes (congés, corrections, absences)                  |
| ExportService     | Génération des rapports CSV et PDF pour le département RH                                             |

Les services sont injectés dans les contrôleurs via l'**autowiring** de Symfony (injection par constructeur), sans couplage direct.

## Modèles (Entity / Repository)

La couche modèle est portée par **Doctrine ORM** :

- **Entités** ( `src/Entity/` ) : représentent les tables de la base de données sous forme de classes PHP annotées. Chaque entité encapsule ses propres règles de validation via les contraintes Symfony ( `@Assert\*` ).

```
src/Entity/
├─ User.php           – utilisateur avec rôles AGENT / RH / ADMIN
├─ Pointage.php       – enregistrement d'arrivée/départ + coordonnées GPS
├─ Site.php           – zone géofencing (coordonnées + rayon en mètres)
├─ Demande.php        – demande de congé / correction / absence
├─ Alerte.php         – alerte automatique liée à un pointage
└─ Document.php       – export généré (CSV ou PDF)
```

- **Repositories** ( `src/Repository/` ) : regroupent toutes les requêtes persistées (DQL / QueryBuilder Doctrine). Le contrôleur ne fait jamais de requête directe : il passe toujours par un repository ou un service.

## Vue (frontend React)

La couche présentation est une **Single Page Application (SPA)** React découplée, accessible sur un domaine/port séparé et consommant exclusivement l'API REST Symfony via des appels HTTP authentifiés par JWT.

```
frontend/src/
├─ pages/             – vues métier (DashboardPage, ValidationPage, RapportsPage...)
├─ components/        – composants réutilisables (UI, calendrier, carte GPS)
├─ services/          – couche d'appel API (axios) – auth, pointage, alerte, demande...
└─ store/             – état global (Zustand) : auth.store.ts, ui.store.ts
```

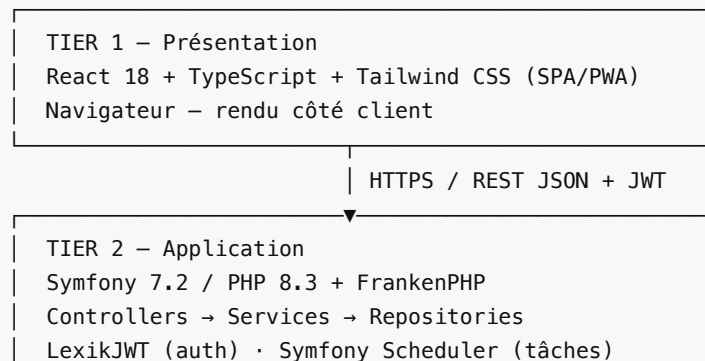
Il n'y a **pas de rendu côté serveur** (pas de templates Twig) : Symfony joue exclusivement le rôle d'API JSON ; React gère intégralement le rendu et la navigation.

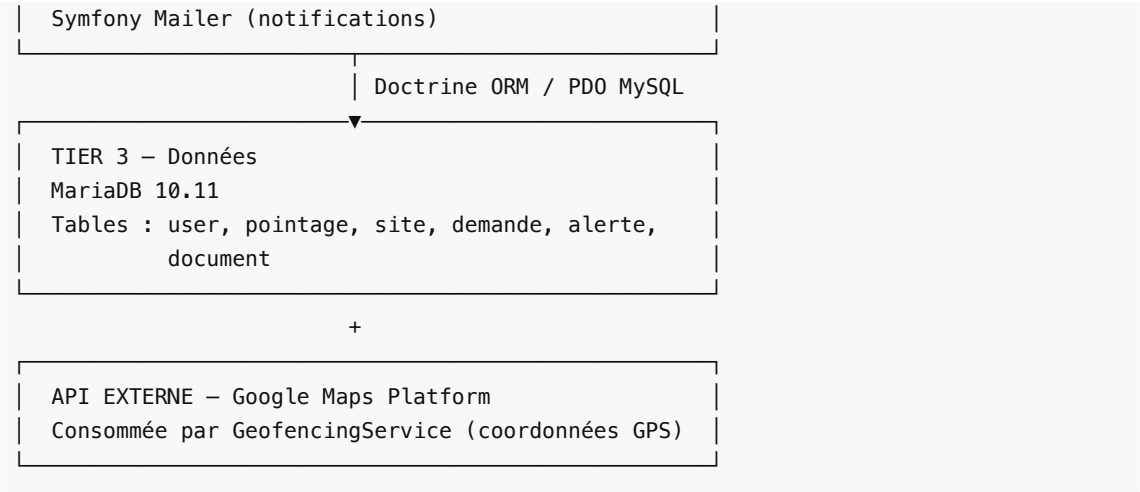
---

## 2. Architecture N-Tiers

### Vue logique (couches)

L'architecture logique est organisée en **4 couches** :



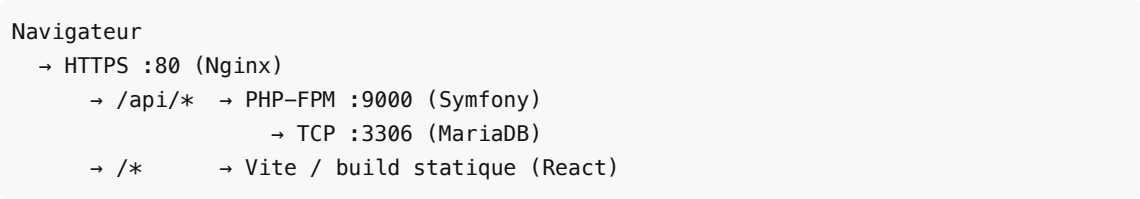


### Vue physique (déploiement Docker)

L'application est **conteneurisée** via Docker Compose avec une séparation stricte des processus :

| Conteneur | Image                       | Rôle physique                                                        | Port exposé    |
|-----------|-----------------------------|----------------------------------------------------------------------|----------------|
| nginx     | nginx:1.25-alpine           | Reverse proxy, terminaison TLS, routage / → frontend, /api → Symfony | 80             |
| api       | php:8.3-fpm-alpine (custom) | FrankenPHP + code Symfony + extensions PDO/GD/Intl                   | interne        |
| frontend  | node:20-alpine (Vite)       | Build React, serveur dev (HMR en dev)                                | interne        |
| db        | mariadb:10.11               | Base de données relationnelle                                        | 3306 (interne) |
| mailpit   | axllent/mailpit             | Capture d'e-mails en développement                                   | 8025           |

### Flux réseau typique :



### Distinction couche logique / tier physique

Il est important de noter que **couche logique** et **tier physique** sont deux notions indépendantes.

Dans notre architecture, les trois tiers logiques (Présentation, Application, Données) s'exécutent sur des **conteneurs distincts**, ce qui constitue une architecture 3-tiers physique. Cependant, en production ces conteneurs pourraient très bien tourner sur **un seul hôte physique** (VM ou serveur dédié) sans que l'architecture logique 3-tiers ne soit remise en cause. La séparation logique garantit l'indépendance des couches ; la séparation physique est une décision d'infrastructure orthogonale.

### 3. Séparation des responsabilités et bonnes pratiques

#### Principes SOLID

| Principe                         | Application concrète dans Horosphere                                                                                                                                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>S</b> — Single Responsibility | Chaque service gère un seul domaine : <code>GeofencingService</code> calcule uniquement les distances GPS ; <code>AlerteService</code> gère uniquement la création et la programmation des alertes. Aucun service ne mélange plusieurs domaines. |
| <b>O</b> — Open/Closed           | Les repositories étendent <code>ServiceEntityRepository</code> de Doctrine sans modifier le comportement parent ; les nouvelles fonctionnalités sont ajoutées par extension.                                                                     |
| <b>L</b> — Liskov                | Les contrôleurs héritent tous de <code>AbstractController</code> Symfony et respectent son contrat d'interface (accès au container, réponses, redirections).                                                                                     |
| <b>I</b> — Interface Segregation | <code>User</code> implémente <code>UserInterface</code> et <code>PasswordAuthenticatedUserInterface</code> de Symfony Security — interfaces minimalistes à responsabilité unique.                                                                |
| <b>D</b> — Dependency Inversion  | L'injection de dépendances Symfony (autowiring) assure que les contrôleurs dépendent d'abstractions (interfaces de services) et non d'implémentations concrètes.                                                                                 |

#### Gestion de la configuration sensible

Toutes les valeurs sensibles sont isolées dans des fichiers d'environnement, jamais codées en dur :

```
.env                - valeurs de développement (non versionné en prod)
.env.example        - template versionné (sans valeurs réelles)
```

Variables concernées : `APP_SECRET` , `DB_PASSWORD` , `JWT_PASSPHRASE` , `GOOGLE_MAPS_API_KEY` , `MAILER_DSN` .

#### Routage et sécurité centralisés

- Le routage est défini dans les attributs PHP `#[Route]` des contrôleurs, centralisé et lisible sans fichier YAML séparé.
- Les règles d'accès sont déclarées dans `config/packages/security.yaml` (firewall `api` avec JWT stateless) et renforcées dans chaque contrôleur via `denyAccessUnlessGranted()` .
- La hiérarchie des rôles ( `ROLE_ADMIN` > `ROLE_RH` > `ROLE_USER` ) est définie une seule fois dans la configuration de sécurité.

#### Validation des données

La validation s'effectue au plus près des données, dans les entités Doctrine via les contraintes Symfony Validator ( `#[Assert\NotBlank]` , `#[Assert\Email]` , `#[Assert\Range]` ...). Les contrôleurs ne contiennent pas de logique de validation métier.

#### Gestion des anomalies et tâches planifiées

`AlerteService` est déclenché de deux manières :

- Synchrone** : lors d'un pointage (détection `HORS_ZONE`, `ECART_HORAIRE`).

- **Asynchrone planifié** : via `symfony/scheduler` , une tâche quotidienne détecte les pointages ouverts depuis plus de 10 heures et génère une alerte `OUBLI_DEPART` .

## Sécurité applicative (OWASP Top 10)

| Risque OWASP                    | Contre-mesure implémentée                                                                                     |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| Injection SQL                   | Doctrine ORM — requêtes paramétrées exclusivement (aucune concaténation SQL)                                  |
| XSS                             | React — échappement automatique du DOM virtuel ; aucun <code>dangerouslySetInnerHTML</code>                   |
| Authentification compromise     | JWT (RS256, clés RSA 4096 bits) + bcrypt/Argon2 pour les mots de passe                                        |
| Accès non autorisé              | <code>denyAccessUnlessGranted()</code> sur chaque endpoint + règles <code>access_control</code>               |
| Exposition de données sensibles | HTTPS obligatoire ; variables d'env hors dépôt Git                                                            |
| CORS                            | <code>nelmio/cors-bundle</code> avec liste blanche des origines autorisées ( <code>CORS_ALLOW_ORIGIN</code> ) |

## 4. Composants externes et bibliothèques

### Backend Symfony — Bundles notables

| Bundle / Package                                 | Version            | Rôle dans l'architecture                                                                                                                                                                                |
|--------------------------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lexik/jwt-authentication-bundle</code>     | <code>^3.1</code>  | Authentification stateless par token JWT (RS256). Remplace les sessions PHP traditionnelles. Le firewall Symfony délègue la vérification du token à ce bundle avant chaque requête <code>api/*</code> . |
| <code>nelmio/cors-bundle</code>                  | <code>^2.4</code>  | Injection des en-têtes CORS dans les réponses Symfony. Nécessaire pour autoriser le frontend React (origine différente) à consommer l'API.                                                              |
| <code>doctrine/doctrine-bundle</code>            | <code>^2.11</code> | ORM relationnel. Gestion du schéma, des migrations et des requêtes DQL via les repositories.                                                                                                            |
| <code>doctrine/doctrine-migrations-bundle</code> | <code>^3.3</code>  | Versionning du schéma de base de données. Chaque évolution structurelle est tracée dans un fichier de migration versionné.                                                                              |
| <code>symfony/scheduler</code>                   | <code>^7.2</code>  | Planification de tâches récurrentes en PHP pur (sans cron système). Utilisé pour la détection quotidienne des oublis de pointage.                                                                       |
| <code>symfony/mailer</code>                      | <code>^7.2</code>  | Envoi d'e-mails transactionnels (notifications RH, alertes). En développement, les e-mails sont interceptés par le conteneur Mailpit.                                                                   |
| <code>symfony/serializer</code>                  | <code>^7.2</code>  | Sérialisation / désérialisation des entités en JSON pour les réponses API.                                                                                                                              |

|                   |      |                                                                                   |
|-------------------|------|-----------------------------------------------------------------------------------|
| symfony/validator | ^7.2 | Validation des données entrantes selon les contraintes déclarées sur les entités. |
|-------------------|------|-----------------------------------------------------------------------------------|

## Frontend React — Packages notables

| Package                   | Version | Rôle dans l'architecture                                                                                                                                                                                   |
|---------------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| react / react-dom         | ^18.3.1 | Framework UI — rendu déclaratif, gestion du Virtual DOM.                                                                                                                                                   |
| react-router-dom          | ^6.26.0 | Routage côté client (SPA). Navigation entre les pages Agent, RH et Admin sans rechargement.                                                                                                                |
| axios                     | ^1.7.2  | Client HTTP. Toutes les requêtes vers l'API Symfony passent par une instance Axios configurée avec l'URL de base et l'intercepteur JWT (injection automatique du header Authorization).                    |
| zustand                   | ^4.5.4  | Gestion d'état global léger. Deux stores : <code>auth.store</code> (token, utilisateur courant, login/logout) et <code>ui.store</code> (modales, notifications). Remplace Redux pour un usage plus simple. |
| @googlemaps/js-api-loader | ^1.16.6 | Chargement dynamique du SDK Google Maps. Utilisé dans le composant de pointage pour afficher la carte et la zone de géofencing du site.                                                                    |
| tailwindcss               | ^3.4.7  | Framework CSS utilitaire. Assure la cohérence visuelle sans feuille de style monolithique.                                                                                                                 |
| date-fns                  | ^3.6.0  | Manipulation et formatage des dates (horodatages des pointages, durées).                                                                                                                                   |
| typescript                | ^5.5.3  | Typage statique du frontend. Assure la cohérence entre les types de l'API et les composants React.                                                                                                         |
| vite                      | ^5.3.4  | Bundler et serveur de développement (HMR). Remplace webpack pour des builds significativement plus rapides.                                                                                                |

## Infrastructure

| Outil                          | Rôle                                                                                                                                                    |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Docker / Docker Compose</b> | Conteneurisation de l'ensemble des services (api, frontend, db, nginx, mailpit). Reproductibilité de l'environnement de développement et de production. |
| <b>Nginx 1.25</b>              | Reverse proxy unique. Redirige le trafic vers Symfony (PHP-FPM) ou vers les assets React selon le chemin d'URL. Gère la terminaison TLS en production.  |
| <b> FrankenPHP</b>             | Runtime PHP moderne (basé sur Caddy). Améliore les performances de PHP-FPM en servant les requêtes directement.                                         |
| <b>MariaDB 10.11</b>           | Moteur de base de données relationnelle compatible MySQL. Choix de stabilité et de performance pour les données transactionnelles de pointage.          |
| <b>Mailpit</b>                 | Serveur SMTP de développement. Capture tous les e-mails sans les envoyer réellement, accessibles via une interface web sur le port 8025.                |

---

## 5. Schémas complémentaires

Les diagrammes UML suivants, disponibles dans le dossier `uml/`, complètent cette description textuelle :

| Fichier                                 | Type                           | Ce qu'il illustre                                                            |
|-----------------------------------------|--------------------------------|------------------------------------------------------------------------------|
| <code>architecture.plantuml</code>      | Diagramme de composants        | Vue déployée complète (conteneurs Docker, flux réseau, APIs externes)        |
| <code>class_diagram.plantuml</code>     | Diagramme de classes           | Entités Doctrine, Repositories, Services et Controllers avec leurs relations |
| <code>use_case.plantuml</code>          | Diagramme de cas d'utilisation | Acteurs (Agent, RH, Admin) et fonctionnalités du système                     |
| <code>sequence_pointage.plantuml</code> | Diagramme de séquence          | Flux complet d'un pointage arrivée/départ avec vérification géofencing       |
| <code>sequence_demande.plantuml</code>  | Diagramme de séquence          | Cycle de vie d'une demande (soumission → validation RH)                      |
| <code>sequence_alerte.plantuml</code>   | Diagramme de séquence          | Détection automatique d'un oubli de pointage par le scheduler                |

---

*Document rédigé dans le cadre du Jalon 4 — Projet fil rouge Horosphere.*